Լաբորատոր աշխատանք №10

1. Աշխատանքի նպատակը

Ուսումնասիրել C++ լեզվում դինամիկ հիշողության կառավարման և լռելյայն կոնստրուկտորի կիրառումը: Սովորել ստեղծել դատարկ և տվյալներով օբյեկտներ, աշխատել դինամիկ զանգվածների հետ, ավելացնել և հեռացնել տարրեր՝ պահպանելով հիշողության անվտանգ կառավարումը:

2. Խնդրի պահանջը

Ստեղծել SavingsPlan դաս, որը պահպանում է ամսական ներդրումների դինամիկ զանգվածը և նպատակային գումարը՝ p:

Դասում սահմանել հետևյալ դաշտերը՝

- double* data – ամսական ներդրումների դինամիկ զանգված:
- int n – զանգվածի տարրերի քանակը:
- double p – նպատակային գումարը:

Իրականացնել հետևյալ մեթոդները՝

1. Լռելյայն կոնստրուկտոր, որը ստեղծում է դատարկ օբյեկտ՝ n=0, data=nullptr, p=1:
2. Պարամետրերով կոնստրուկտոր, որը ստեղծում է տվյալներով օբյեկտ:
3. Դեստրուկտոր՝ դինամիկ հիշողությունն ազատելու համար:
4. append(value)՝ նոր ներդրում ավելացնելու համար:
5. removeAt(index)՝ նշված ինդեքսով ներդրումը հեռացնելու համար:
6. print()՝ զանգվածի տարրերը արտածելու համար:
7. Ընդհանուր խնայողության հաշվարկ:
8. Նպատակային գումարին հասնելու առաջին ամսվա ինդեքսի որոշում:
9. Միջին ներդրումից փոքր ներդրումների քանակի հաշվարկ:

Թույլատրելի են միայն ոչ բացասական ներդրումները և դրական նպատակային գումարը: Սկզբնական զանգվածի չափը պետք է լինի 3-ից 30:

3. Առաջադրանքի լուծումը (C++ կոդ)

```
#include <iostream>
```

```
#include <iomanip>
#include <cmath>
using namespace std;

class SavingsPlan {
private:
   double* data;
   int n;
   double p;

public:
   // Լռելյայն կոնստրուկտոր
   SavingsPlan() {
      data = nullptr;
      n = 0;
      p = 1;
   }

   // Պարամետրերով կոնստրուկտոր
   SavingsPlan(const double arr[], int size, double target) {
      data = nullptr;
      n = 0;
      p = 1;

      if (size < 3 || size > 30 ||
         !isfinite(target) || target <= 0)
         return;

      for (int i = 0; i < size; i++) {
```

```
        if (!isfinite(arr[i]) || arr[i] < 0)
            return;
    }

    data = new double[size];

    for (int i = 0; i < size; i++)
        data[i] = arr[i];

    n = size;
    p = target;
}

// Դեստրուկտոր
~SavingsPlan() {
    delete[] data;
}

// Նոր ներդրման ավելացում
bool append(double value) {
    if (!isfinite(value) || value < 0 || n >= 30)
        return false;

    double* temp = new double[n + 1];

    for (int i = 0; i < n; i++)
        temp[i] = data[i];

    temp[n] = value;
```

```
    delete[] data;
    data = temp;
    n++;

    return true;
}

// Ներդրման հեռացում
bool removeAt(int index) {
    if (index < 0 || index >= n)
        return false;

    double* temp = nullptr;

    if (n > 1)
        temp = new double[n - 1];

    for (int i = 0, j = 0; i < n; i++) {
        if (i != index)
            temp[j++] = data[i];
    }

    delete[] data;
    data = temp;
    n--;

    return true;
}
```

```
// Զանգվածի արտածում
void print() const {
    cout << "Ներդրումներ: ";

    if (n == 0)
        cout << "Դատարկ";
    else {
        for (int i = 0; i < n; i++)
            cout << data[i] << " ";
    }

    cout << "\nՉափ: " << n;
    cout << "\nՆպատակային գումար: " << p << endl;
}

// Ընդհանուր խնայողություն
double totalSavings() const {
    double sum = 0;

    for (int i = 0; i < n; i++)
        sum += data[i];

    return sum;
}

// Նպատակային գումարին հասնելու առաջին ինդեքսը
int firstTargetMonth() const {
    double sum = 0;
```

```
        for (int i = 0; i < n; i++) {
            sum += data[i];

            if (sum >= p - 1e-9)
                return i;
        }

        return -1;
    }

    // Միջինից փոքր ներդրումների քանակ
    int countBelowAverage() const {
        if (n == 0)
            return 0;

        double average = totalSavings() / n;
        int count = 0;

        for (int i = 0; i < n; i++) {
            if (data[i] < average - 1e-9)
                count++;
        }

        return count;
    }
};

int main() {
```

```
cout << fixed << setprecision(2);

double payments[] = {10000, 15000, 8000, 20000};
int size = 4;
double target = 30000;

// Առաջին օբյեկտ՝ լռելյայն կոնստրուկտորով
SavingsPlan plan1;

cout << "Առաջին օբյեկտը:\n";
plan1.print();

// Տարրերի ավելացում
for (int i = 0; i < size; i++) {
    plan1.append(payments[i]);
}

cout << "\nԱռաջին օբյեկտը լրացնելուց հետո:\n";
plan1.print();

// Երկրորդ օբյեկտ՝ պարամետրերով
SavingsPlan plan2(payments, size, target);

cout << "\nԵրկրորդ օբյեկտը:\n";
plan2.print();

// Առաջին օբյեկտից հեռացնել ինդեքս 1-ը
plan1.removeAt(1);
```

```
    cout << "\nԱռաջին օբյեկտից ինդեքս 1-ը հեռացնելուց հետո:\n";
    plan1.print();

    // Հաշվարկների արտածում
    auto calculate = [](const SavingsPlan& plan) {
        cout << "\nԸնդհանուր խնայողություն: "
             << plan.totalSavings() << endl;

        int month = plan.firstTargetMonth();

        if (month == -1)
            cout << "Նպատակային գումարը չի հասել\n";
        else
            cout << "Նպատակին հասնելու ինդեքս: "
                 << month << endl;

        cout << "Միջինից փոքր ներդրումների քանակ: "
             << plan.countBelowAverage() << endl;
    };

    cout << "\nԱռաջին օբյեկտի հաշվարկները:";
    calculate(plan1);

    cout << "\nԵրկրորդ օբյեկտի հաշվարկները:";
    calculate(plan2);

    return 0;
}
```